Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil
DIAGNOSTIC ACCOMPAGNEMENT FORMATION

SYMFONY – CRUD RECETTE – PART3

Introduction

L'affichage de nos recettes fonctionne nous allons maintenant faire notre create

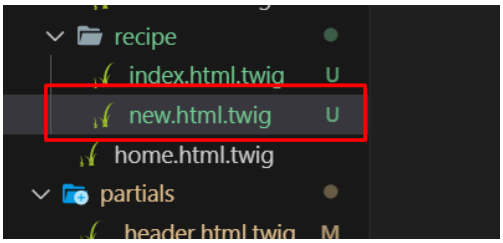
I) La public function new

```
33          return $this->render('pages/recipe/index.html.twig', [
34              'recipes' => $recipes
35          ]);
36      }
37
38      #[Route('/recette/creation', name: 'app_recipe_new', methods:['GET','POST'])]
39      public function new() : Response
40      {
41          return $this->render('pages/recipe/new.html.twig');
42      }
```

Nous allons donc, dans notre controller RecipeController et à la suite de notre function index créer la méthod ci-dessus

II) La view

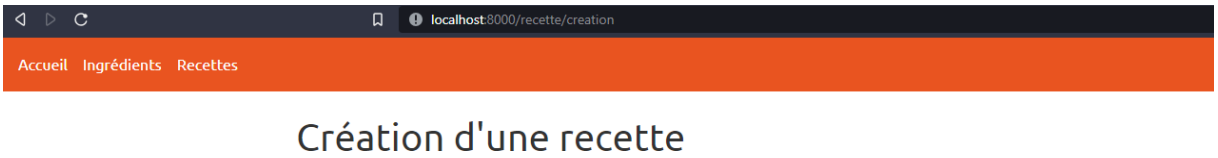
Nous reviendrons par la suite agrémenter le code pour faire le create mais passons à la view et dans le dossier Recipe il faut créer le fichier new.html.twig comme ceci :



Préparons notre view de cette façon le formulaire viendra par la suite. Nous allons suivre la même procédure que lorsque nous avons fait notre create pour l'ingrédient

```
templates > pages > recipe > new.html.twig
1  {% extends "base.html.twig" %}
2
3  {% block body %}
4      <div class="container">
5
6          <h1 class="mt-4">Création d'une recette</h1>
7
8  {% endblock %}
```

Dans notre navigateur nous avons donc ce visuel



III) Le Form

Faisons maintenant notre formulaire d'ajout avec la commande « php bin/console make:form »

```
PS C:\Users\alexa\OneDrive\Bureau\workspace\symfonytuto\firstproject> php bin/console make:form

  The name of the form class (e.g. GrumpyChefType):
  >
```

Donnons-lui un nom et devinez quoi nous allons l'appeler RecipeType

```
  The name of the form class (e.g. GrumpyChefType):
  > Recipetype

  The name of Entity or fully qualified model class name that the new form will be bound to (empty for none):
  > Recipe
```

Et il se basera sur l'entity Recipe

Notre RecipeType est donc créé et regardons ce qu'il nous affiche :

```
class RecipeType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options):
    {
        $builder
            ->add('name')
            ->add('time')
            ->add('nbPeople')
            ->add('difficulty')
            ->add('description')
            ->add('price')
            ->add('isFavorite')
            ->add('createdAt')
            ->add('updatedAt')
            ->add('ingredients')
        ;
    }
```

Nous voyons qu'il a repris toutes les propriétés de notre entity Recipe et c'est une bonne chose

Il faut supprimer le createdAt et le udatedAt car ils se créent automatiquement et nous n'avons pas besoin de les faire intervenir dans notre formulaire

Nous allons aussi ajouter un submit

Nous avons désormais cela :

```
{
    public function buildForm(FormBuilderInterfac
    {
        $builder
            ->add('name')
            ->add('time')
            ->add('nbPeople')
            ->add('difficulty')
            ->add('description')
            ->add('price')
            ->add('isFavorite')
            ->add('ingredients')
            ->add('submit')
        ;
    }
```

IV) Implémentation du Formulaire dans la vue

Comme dans les chapitres précédents nous allons implémenter le formulaire dans la view.

Retournons dans notre fichier new.html.twig de notre dossier Recipe et mettons le formulaire de la façon la plus basique pour le moment comme ceci {{form(form)}}

```
templates > pages > recipe > new.html.twig
1  {% extends "base.html.twig" %}
2
3  {% block body %}
4      <div class="container">
5
6          <h1 class="mt-4">Création d'une recette</h1>
7
8          {{form(form)}}
9
10 {% endblock %}
```

Retournons dans notre controller car il faut dire au controller qu'il nous le crée

```
35          return $this->render('pages/recipe/index.html.twig', [
36              'recipes' => $recipes
37          ]);
38      }
39
40      #[Route('/recette/creation', name: 'app_recipe_new', methods:['GET','POST'])]
41      public function new() : Response
42      {
43          $recipe = new Recipe();
44          $form = $this->createForm(RecipeType::class, $recipe);
45
46          return $this->render('pages/recipe/new.html.twig',[
47              'form' => $form->createView(),
48          ]);
49
50      }
51  }
```

Ensuite il faut que l'on modifie le submit dans notre RecipeType

```
public function buildForm(FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('name')
        ->add('time')
        ->add('nbPeople')
        ->add('difficulty')
        ->add('description')
        ->add('price')
        ->add('isFavorite')
        ->add('ingredients')
        ->add('submit',SubmitType::class)
    ;
}
```

Dans notre entity Ingredient il faut faire la method __tostring() qui nous retournera le name de l'ingredient car il ne sait pas actuellement au niveau de l'ingrédient ce qu'il doit nous retourner

```
80      }
81
82      public function setCreatedAt(\DateTimeImmutable $createdAt): self
83      {
84          $this->createdAt = $createdAt;
85
86          return $this;
87      }
88
89      public function __toString()
90      {
91          return $this->name;        You, 5 minutes ago • Uncommitted change
92      }
93  }
94
```

Regardons maintenant dans notre navigateur ce qu'il nous affiche

Le formulaire s'affiche alors effectivement niveau esthétique ce sera à travailler. Nous voyons que nos ingrédient se mettent sous forme de liste dans un menu select. On verra pour afficher les ingrédients dans des checkbox

V) Le RecypeType

Occupons nous de notre RecypeType comme nous avons pu le faire dans IngredientType

```
->add('name',TextType::class,[
    'attr'=> [
        'class'=>'form-control',
        'minlength'=>2,
        'maxlength'=>50
    ],
    'label'=>'Nom',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\Length(['min'=>2,'max'=>50]),
        new Assert\NotBlank()
    ]
])
```

```
->add('time',IntegerType::class,[
    'attr'=>[
        'class'=>'form-control',
        'min'=>1,
        'max'=>1440
    ],
    'label'=>'temps(en minutes)',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\Positive(),
        new Assert\LessThan(1441)
    ]
])
```

```
->add('nbPeople',IntegerType::class,[
    'attr'=>[
        'class'=>'form-control',
        'min'=>1,
        'max'=>50
    ],
    'label'=>'nombre de personnes',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\Positive(),
        new Assert\LessThan(51)
    ]
    ]
)
```

```
->add('difficulty',RangeType::class,[
    'attr'=>[
        'class'=>'form-range',
        'min'=>1,
        'max'=>5
    ],
    'label'=>'Difficulté',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\Positive(),
        new Assert\LessThan(5)
    ]
])
```

```
->add('description',TextareaType::class,[
    'attr'=>[
        'class'=>'form-control',
        'min'=>1,
        'max'=>5
    ],
    'label'=>'Description',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\NotBlank()
    ]
])
```

```
->add('price',MoneyType::class,[
    'attr'= [
        'class'=>'form-control',
    ],
    'label'=>'Prix ',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\Positive(),
        new Assert\LessThan(1001)
    ]
])
```

```
->add('isFavorite',CheckboxType::class,[
    'attr'= [
        'class'=>'form-control',
    ],
    'label'=>'Favori ? ',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'constraints'=>[
        new Assert\NotNull()
    ]
])
```

```
->add('ingredients',EntityType::class,[
    'label'=>'Les ingrédients ',
    'label_attr'=>[
        'class' => 'form-label mt-4'
    ],
    'class'=>Ingredient::class,
    'query_builder' => function (IngredientRepository $r) {
        return $r->createQueryBuilder('i')
            ->orderBy('i.name', 'ASC');
    },
    'choice_label'=>'name',
    'multiple'=>'true',
    'expanded'=>'true',
])
```

```
137            ->add('submit',SubmitType::class,[
138                'attr'=>['
139                class'=>'btn btn-primary mt-4'
140                ],
141                'label'=> 'créer ma recette'
142            ]);
```

Pour les ingredients dans le RecipeType nous disons que

- c'est un EntityType
- On défit quel entity et ici c'est Ingredient
- Nous faisons une QueryBuilder (requête)
- Nous mettons multiple à true car nous voulons faire une sélection multiple sur les ingrédients
- Nous mettons expanded car nous voulons un affichage de checkbox

Toute est effectivement marqué dans la [doc de symfony]

If you want to create a custom query to use when fetching the entities (e.g. you only want to return some entities, or need to order them), use the query_builder option:

```
1  use App\Entity\User;
2  use Doctrine\ORM\EntityRepository;
3  use Symfony\Bridge\Doctrine\Form\Type\EntityType;
4  // ...
5
6  $builder->add('users', EntityType::class, [
7      'class' => User::class,
8      'query_builder' => function (EntityRepository $er) {
9          return $er->createQueryBuilder('u')
10             ->orderBy('u.username', 'ASC');
11     },
12     'choice_label' => 'username',
13 ]);
```

Using form collections may result in making too many database requests to fetch related

Les-Tilleuls.coop
basecom
Become certified from hom
blackfire.io
Check Code Performance in Test, Staging & Production

- Using Choices
- Select Tag, Checkboxes or Radio Buttons
- Field Options
  - choice_label
  - class
  - em
  - query_builder
- Overridden Options
  - choice_name
  - choice_value
  - choices
  - data_class
- Inherited Options
  - choice_attr
  - choice_translation_domain
  - expanded
  - group_by

The `entity` type has just one required option: the entity which should be listed inside the choice field:

```
use App\Entity\User;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;
// ...

$builder->add('users', EntityType::class, [
    // looks for choices from this entity
    'class' => User::class,

    // uses the User.username property as the visible option string
    'choice_label' => 'username',

    // used to render a select box, check boxes or radios
    // 'multiple' => true,
    // 'expanded' => true,
]);
```

This will build a `select` drop-down containing *all* of the `User` objects in the database. To render radio buttons or checkboxes instead, change the multiple and expanded options.

Using a Custom Query for the Entities

shopware
Les-Tilleuls.coop
basecom
Become certified from home
blackfire.io
Check Code Performance in Dev, Test, Staging & Production

Pour les autres champs c'est dans le même esprit que dans IngredientType

VI) Le new.html.twig

Retournons dans le new.html.twig de notre dossier Recipe et travaillons le de la même façon que la view du formulaire de création des ingrédients

```
6       <h1 class="mt-4">Création d'un ingrédient</h1>
7
8       {{ form_start(form) }}
9
10          <div class="form-group">
11              {{ form_label(form.name) }}
12              {{ form_widget(form.name) }}
13              <div class="form-error">
14                  {{ form_errors(form.name) }}
15              </div>
16          </div>
17          <div class="form-group">
18              {{ form_label(form.nbPeople) }}
19              {{ form_widget(form.nbPeople) }}
20              <div class="form-error">
21                  {{ form_errors(form.nbPeople) }}
22              </div>
23          </div>
```

```
24          <div class="form-group">
25              {{ form_label(form.time) }}
26              {{ form_widget(form.time) }}
27              <div class="form-error">
28                  {{ form_errors(form.time) }}
29              </div>
30          </div>
31          <div class="form-group">
32              {{ form_label(form.difficulty) }}
33              {{ form_widget(form.difficulty) }}
34              <div class="form-error">
35                  {{ form_errors(form.difficulty) }}
36              </div>
37          </div>
38          <div class="form-group">
39              {{ form_label(form.description) }}
40              {{ form_widget(form.description) }}
41              <div class="form-error">
42                  {{ form_errors(form.description) }}
43              </div>
44          </div>
```

```
45          <div class="form-group">
46              {{ form_label(form.price) }}
47              {{ form_widget(form.price) }}
48              <div class="form-error">
49                  {{ form_errors(form.price) }}
50              </div>
51          </div>
52          <div class="form-group">
53              {{ form_label(form.isFavorite) }}
54              {{ form_widget(form.isFavorite) }}
55              <div class="form-error">
56                  {{ form_errors(form.isFavorite) }}
57              </div>
58          </div>
59
60          {{form_row(form.ingredients)}}
61
62      {{ form_end(form) }}
63  </div>
64
65  {% endblock %}
```

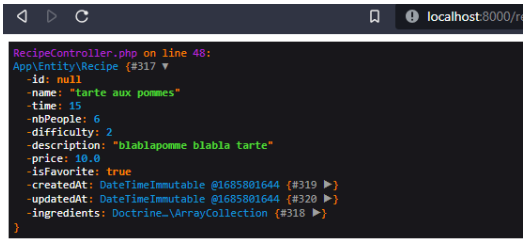
VII) Finissons le controller

Nous allons faire comme dans le controller Ingredient controller pour la function new

```
39
40      #[Route('/recette/creation', name: 'app_recipe_new', methods:['GET','POST'])]
41      public function new(Request $request) : Response
42      {
43          $recipe = new Recipe();
44          $form = $this->createForm(RecipeType::class, $recipe);
45
46          $form->handleRequest($request);
47          if($form->isSubmitted() && $form->isValid()){
48              dd($form->getData());
49          }
50
51          return $this->render('pages/recipe/new.html.twig',[
52              'form' => $form->createView(),
53          ]);
54
55      }
```

Nous faisons un dd pour voir ce que nous récupérons de notre formulaire une fois soumis et validé

Regardons le retour dans le navigateur



Nous pouvons nous amuser à déplier les ingrédients et nous voyons que l'on a bien le nom des ingrédients avec son prix affiché

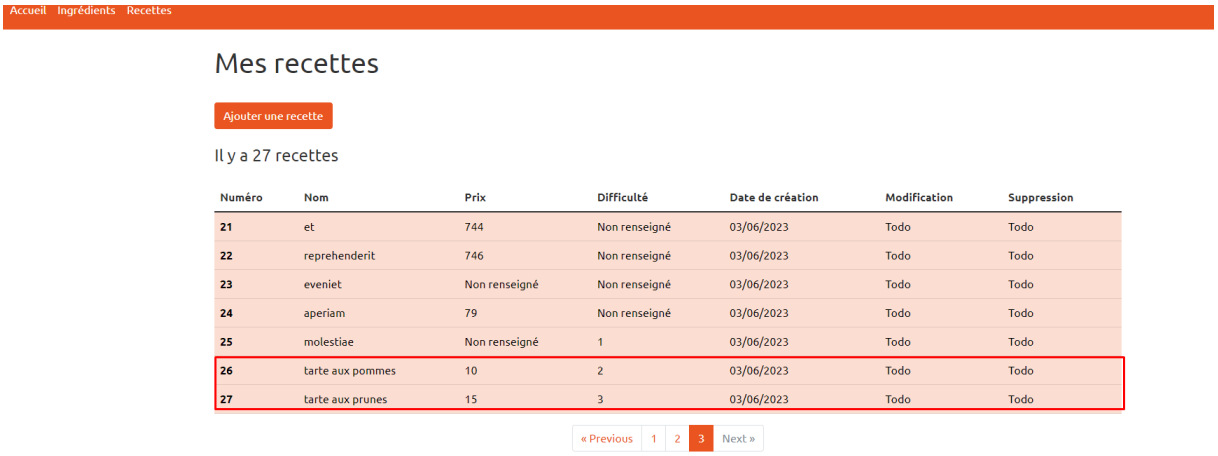
Nous récupérons vraiment tout ce que l'on a besoin

Finissons donc le controller en faisant le persist le flush et en mettant le message addFlash comme ceci :

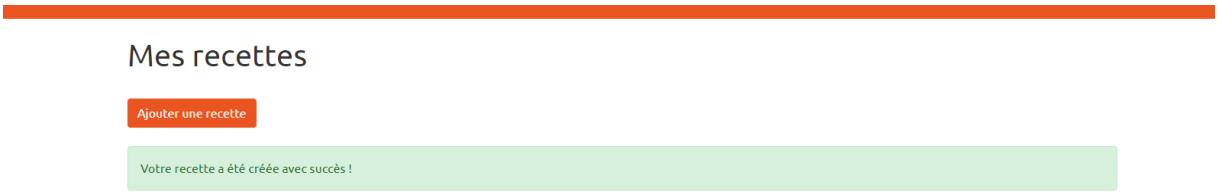
```
47          $form->handleRequest($request);
48          if($form->isSubmitted() && $form->isValid()){
49              $recipe =$form->getData();
50
51              $manager->persist($recipe);
52              $manager->flush();
53
54              $this->addFlash(
55                  'success',
56                  'Votre recette a été créée avec succès !'
57              );
58
59              return $this->redirectToRoute('app_recipe');
60
61          }
62
63          return $this->render('pages/recipe/new.html.twig',[
64              'form' => $form->createView(),
65          ]);
66
```

N'oubliez pas également la redirection « return $this->redirectToRoute('app_recipe ') »

Au niveau du visuel nous avons bien nos recettes qui se mettent en bdd et nous les avons dans notre tableau en retour avec notre message flash



Numéro	Nom	Prix	Difficulté	Date de création	Modification	Suppression
21	et	744	Non renseigné	03/06/2023	Todo	Todo
22	reprehenderit	746	Non renseigné	03/06/2023	Todo	Todo
23	eveniet	Non renseigné	Non renseigné	03/06/2023	Todo	Todo
24	aperiam	79	Non renseigné	03/06/2023	Todo	Todo
25	molestiae	Non renseigné	1	03/06/2023	Todo	Todo
26	tarte aux pommes	10	2	03/06/2023	Todo	Todo
27	tarte aux prunes	15	3	03/06/2023	Todo	Todo



Nous arrivons donc à créer notre recette nous allons faire la suite dans le prochain chapitre c’est-à-dire la modification et la suppression de la recette